

Section A: Concept Application Answers

S1: Missing Values and Column Types

To handle the 14% missing values in the 'delivery_time_mins' column, imputation is preferred over dropping rows. Dropping 14% of the data could lead to a significant loss of information and reduce the dataset size considerably. If the delivery times are highly skewed, using the median is the best approach as it is robust to outliers. If the data is normally distributed, the mean could be used. To fix the 'customer_rating' column type before any analysis is performed, you would use Pandas'

```
pd.to_numeric(df['customer_rating'], errors='coerce') or  
df['customer_rating'].astype(float).
```

The `errors='coerce'` argument is especially helpful because it safely handles any unparseable strings by turning them into NaNs.

S2: NumPy Boolean Indexing and Stats

A boolean mask is an array of boolean values (True/False) that corresponds to the elements of another array based on a specific condition. By calculating the threshold using statistical functions (`threshold = np.mean(orders) + 1.5 * np.std(orders)`), we create a mask: `mask = orders > threshold`. This mask can then be used to filter the revenue array directly: `surge_revenue = revenue[mask]`. Element-wise array operations make this approach significantly faster than looping because NumPy delegates these vectorized operations to highly optimized, pre-compiled C code, entirely bypassing Python's slow interpreter loop.

S3: EDA Business Questions

(a) Delivery time vs customer rating: This requires Bivariate analysis. A Seaborn scatter plot (`sns.scatterplot`) is best to visualize the relationship between two continuous numeric variables, as it can clearly reveal correlation, clustering, or patterns. A line plot is inappropriate because it suggests a sequential or time-series relationship between the data points.

(b) Avg order value distribution across restaurant categories: This requires Bivariate analysis. A Seaborn box plot (`sns.boxplot`) or violin plot is ideal for visualizing the distribution of a continuous numeric variable across discrete categories. It clearly shows medians, quartiles, and outliers within each category, which a simple line plot cannot represent effectively.

S4: SQL Joins

A LEFT JOIN (with the `restaurants` table on the left and `orders` on the right) must be used. This ensures all restaurants are included in the final report, even if they have no

matching records in the orders table (their calculated revenue will simply evaluate to NULL or 0). If an INNER JOIN were used, restaurants with zero orders would be completely excluded from the report, because an INNER JOIN strictly requires matching keys to exist in both tables to return a row.

S5: SQL Window Functions

To rank agents within each city, use the `RANK()` or `DENSE_RANK()` window function: `RANK() OVER (PARTITION BY city ORDER BY completed_deliveries DESC)`. To retrieve the previous row's value for comparison, use the `LAG()` function:

`LAG(completed_deliveries) OVER (PARTITION BY city ORDER BY completed_deliveries DESC)`. Using window functions with `PARTITION BY` produces more accurate and maintainable results than a `GROUP BY` with a self-join because window functions calculate aggregates while preserving the original row-level details. Self-joins require complex, verbose syntax that is harder to read, harder to debug, and usually slower to execute.

S6: CTE vs Nested Subquery

A CTE (Common Table Expression or `WITH` clause) greatly improves readability by moving the subquery logic to the top of the script, breaking the problem down into logical, sequential steps. It also allows the intermediate result (the top 5 categories) to be reused multiple times in the main query if needed and simplifies debugging since the CTE can be tested independently. A nested subquery, on the other hand, can be deeply indented and harder to read. However, a nested subquery might be preferred in situations where the logic is very simple, the intermediate step is only used once, and the query needs to be extremely compact for a one-off inline filter.